HarmoRAG: Efficient Query-Adaptive Execution for RAG Serving

Abstract

Retrieval-Augmented Generation (RAG) grounds large language models in external knowledge by coupling vector retrieval with evidence-based generation. This coupled execution incurs retrieval and context-processing costs that depend on the search effort and evidence required by each query, making uniform search budgets (*nprobe*) and context sizes (top-*k*) inefficient. Recent approaches adapt these configurations to individual queries but largely rely on one-shot decisions, without exploiting how retrieval evolves during execution. Consequently, runtime retrieval dynamics—including search convergence, early evidence availability, and asynchronous per-query progress—remain largely unexploited across the retrieval–generation pipeline. In this paper, we present HarmoRAG, a query-adaptive RAG serving system that coordinates retrieval and generation according to runtime retrieval progress. HarmoRAG introduces *rank-aware early termination*, which identifies search convergence from evolving top-ranked results and terminates redundant probing, and *speculative incremental augmentation* to initiate generation from early evidence and progressively extend the context with prefix KV-cache reuse. It further employs *cross-stage continuous batching* to propagate asynchronous per-query progress across the RAG pipeline, reducing batch-tail blocking while maintaining batching efficiency. Evaluations across diverse RAG workloads show that HarmoRAG achieves up to 1.69× the peak throughput and a 4.06× end-to-end latency speedup over state-of-the-art RAG serving systems while preserving answer quality.

1 Introduction

Large Language Models (LLMs) have revolutionized knowledge-intensive applications by demonstrating remarkable capabilities in language understanding, reasoning, and generation through large-scale pre-training [5, 11, 24, 38]. However, the knowledge embedded in LLM parameters is inherently static, limiting their ability to incorporate evolving information, adapt to specialized domains, and provide verifiable evidence without expensive retraining [8, 45]. These limitations stem from the entanglement of knowledge storage and model parameters, where updating external knowledge requires modifying the entire model rather than directly accessing dynamic data sources. *Retrieval-Augmented Generation* (RAG) addresses this challenge by decoupling knowledge storage from model parameters, enabling LLMs to dynamically retrieve relevant information from external sources and generate responses grounded in fresh and domain-specific evidence [30]. By combining the reasoning capability of LLMs with the scalability of external knowledge systems, RAG has become a promising paradigm for deploying reliable and continuously evolving intelligent applications.

Unlike conventional LLM inference, RAG introduces external knowledge retrieval into the serving path, forming a multi-stage execution pipeline that couples retrieval with generation. As illustrated in Figure 1, the retrieval stage searches external knowledge sources and selects top-*k* relevant documents, which are subsequently incorporated into the LLM context for evidence-grounded generation [15, 25]. This retrieval-augmented execution introduces

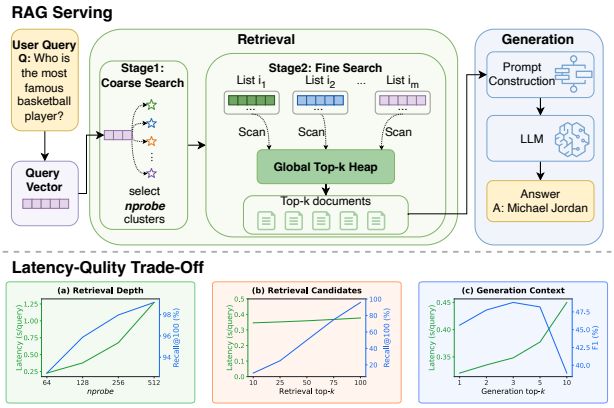


Figure 1: RAG serving and its latency–quality trade-offs.

additional computation beyond conventional LLM inference. The retrieval process incurs explicit search overhead, while its output directly affects the computational demand of the generation stage. In particular, the retrieved documents determine the input sequence length and, consequently, the prefill computation. End-to-end RAG serving efficiency therefore depends on both retrieval effort and the downstream cost of processing retrieved contexts.

These costs are governed by several key execution parameters along the RAG pipeline, each exhibiting a distinct latency–quality trade-off. RAG retrieval commonly relies on *Approximate Nearest Neighbor Search* (ANNS), such as partition-based IVF and graph-based HNSW, whose search effort and output are controlled by parameters such as *nprobe* and retrieval top-*k* [9, 37]. As shown in Figure 1(a), *nprobe* controls the search depth of IVF by specifying the number of index partitions to probe. Increasing *nprobe* expands the search space and improves retrieval recall, but incurs higher search latency. Retrieval top-*k* instead controls how many candidate documents are exposed to downstream generation. Figure 1(b) shows that increasing retrieval top-*k* improves recall with little additional retrieval latency, while making more retrieved evidence available for generation. Generation top-*k* then determines how much of this evidence is incorporated into the LLM context. As shown in Figure 1(c), increasing generation top-*k* enlarges the input context and raises inference latency, whereas generation quality improves only up to a moderate value and may even degrade as less relevant evidence is introduced. Together, these trade-offs reveal diminishing returns across the RAG pipeline, where execution cost continues to increase after quality gains begin to saturate.

In practice, RAG queries vary substantially in the retrieval effort and contextual evidence required for high-quality generation [46, 57], making uniform execution configurations inherently inefficient. Existing RAG serving optimizations, however, largely assume a prescribed retrieval and generation configuration and focus on executing it more efficiently. Vector-search systems improve ANNS indexing and query execution [12, 49], while RAGCache [21], CacheBlend [58], and FusionRAG [47] reduce context-processing overhead through caching, selective recomputation, and

cross-chunk KV reuse. Systems such as RAGO [20], HedraRAG [14] and TeleRAG [33] co-optimize retrieval and LLM inference through resource placement, scheduling, and pipelining to improve end-to-end serving efficiency. Collectively, these techniques make a given RAG execution more efficient, but do not determine how much retrieval and context processing each query actually requires.

Recent work has begun to address this limitation through query-adaptive RAG execution. However, most existing approaches make query-specific adaptation decisions without continuously exploiting how retrieval evolves at runtime. Adaptive ANNS systems, including LAET [31], Tao [55], DARTH [6], and Recall-Aware ANNS [13], adjust search effort for individual queries. Their termination decisions typically rely on trained predictors, incurring additional training costs and requiring retraining as the workload or index evolves. Self-RAG [1], Adaptive-RAG [19] and METIS [42] instead use LLM-derived signals to to adapt RAG execution at the generation, workflow, and system-configuration levels, respectively. These online decisions improve flexibility but add inference and sequential dependencies to the critical path. Adaptive-k [44] derives a query-specific LLM context size from the similarity distribution of retrieved candidates, but determines k only after a fixed candidate set has been retrieved, thereby retaining strict retrieval–generation serialization. These approaches adapt query execution primarily through discrete, intra-stage decisions. Beyond their decision logic, query-level adaptations also reshape the execution dynamics of RAG serving. Varying search effort causes requests within a batch to complete retrieval at different times, whereas conventional RAG pipelines retain batch-synchronous retrieval semantics. The resulting synchronization forces early-completing requests to wait for stragglers before entering generation, delaying generation admission and reducing cross-stage execution efficiency.

These limitations motivate a closer examination of retrieval dynamics and their implications for downstream execution. Across representative RAG workloads, including NQ [28], SQuAD [41], and HotpotQA [57], we reveals three key properties. First, retrieval quality is highly sensitive to changes among top-ranked results, while becoming increasingly stable once the leading ranks converge. This rank-sensitive convergence makes high-rank evolution an informative runtime signal of diminishing search utility. Second, high-ranked evidence exhibits strong persistence across increasing search depths, with passages identified early often remaining among the final top results. This persistence allows useful evidence to become available before retrieval completes, separating evidence readiness from the final retrieval state. Third, query-adaptive retrieval creates asynchronous per-query progress, amplifying intra-batch execution skew as queries converge at different search depths. Under batch-level synchronization, such progress remains hidden until the retrieval tail completes, preventing per-query search savings from translating into earlier generation admission. Together, these observations reveal opportunities to adapt retrieval, evidence consumption, and cross-stage execution at query granularity.

In this paper, we present HarmoRAG, a system for efficient query-adaptive RAG serving that leverages runtime retrieval progress to adapt execution throughout the retrieval–generation pipeline. HarmoRAG builds on three key techniques. (1) *Rank-aware early termination* (§4.1) eliminates redundant retrieval by detecting search convergence directly from the evolution of top-ranked results. It assigns greater importance to higher-ranked documents and tracks their weighted rank variations during search, terminating retrieval once the ranking stabilizes. (2) *Speculative incremental augmentation* (§4.2) exploits the persistence of early retrieved evidence to decouple generation from retrieval completion. HarmoRAG speculatively initiates generation with stable top-ranked evidence while retrieval continues, and incrementally incorporates newly retrieved evidence when needed, reusing the existing prefix KV cache to avoid redundant prefill. (3) *Cross-stage continuous batching* (§4.3) eliminates batch-tail blocking under query-adaptive retrieval. HarmoRAG decomposes retrieval into resumable per-query execution and dynamically schedules retrieval and generation batches, allowing completed queries to proceed immediately and enabling fine-grained overlap between CPU retrieval and GPU generation. These techniques shift RAG serving from globally configured execution to efficient query-adaptive execution.

We implement HarmoRAG with approximately 2.4K lines of Python and 0.9K lines of C++ code on top of Faiss (v1.8.0) [9] for vector retrieval and vLLM (v0.11.2) [29] for LLM serving, requiring only lightweight extensions to existing retrieval and serving backends. Evaluations across diverse RAG workloads, QA datasets, model scales, and request loads show that HarmoRAG consistently improves end-to-end serving efficiency while maintaining retrieval recall and answer quality, demonstrating that its query-adaptive execution remains effective across a broad range of RAG settings.

Our contributions are summarized as follows:

- We characterize RAG serving and uncover three key properties—retrieval convergence, early evidence availability, and asynchronous query progress—that expose new opportunities for efficient query-adaptive execution.
- We present HarmoRAG, an efficient query-adaptive RAG serving system that integrates rank-aware early termination, speculative incremental augmentation, and cross-stage continuous batching for end-to-end execution adaptation.
- We evaluate HarmoRAG across representative RAG workloads and system configurations. HarmoRAG achieves up to 1.69× higher throughput and a 4.06× end-to-end latency speedup while maintaining comparable answer quality.

2 Background

2.1 Large-Scale Vector Retrieval

RAG retrieval is commonly formulated as vector similarity search over an embedding index. Given a query embedding $q \in \mathbb{R}^D$ and a collection $X = x_1, x_2, \ldots, x_N$, where $x_i \in \mathbb{R}^D$, the retriever identifies the top-k vectors most similar to q under a distance metric such as cosine similarity, inner product, or Euclidean distance [23, 25]. The corresponding documents are then supplied as contextual evidence for downstream generation. Finding the exact top-k requires comparing q against all N vectors, incurring $O(ND)$ computation and becoming prohibitively expensive at scale. Large-scale RAG systems therefore rely on *Approximate Nearest Neighbor Search* (ANNS), which restricts distance evaluation to a subset of indexed vectors [3, 50]. ANNS reduces search cost at the expense of retrieval quality, commonly measured by Recall@k, the fraction of exact top-k neighbors recovered by the approximate search.

ANNS indexes expose index-specific parameters to control search effort, with HNSW using *efSearch* to regulate graph traversal and IVF using *nprobe* to determine the number of partitions searched [23, 37]. Increasing these parameters generally improves recall at higher computation cost. Among these indexes, we focus on IVF because its partition-wise execution exposes how retrieval results evolve as the search scope expands. IVF partitions the embedding space into inverted lists associated with coarse centroids. At query time, the centroids are ranked by proximity to the query and their lists are probed in order. Each probe contributes new candidates to a retrieval top-k, progressively refining its membership and ranking. IVF retrieval can therefore be viewed as a sequence of intermediate ranking states at partition boundaries, making the evolution of top-ranked results directly observable during search.

Conventional RAG retrieval, however, does not expose these intermediate states to downstream execution. Search parameters such as *nprobe* are typically configured at the workload level to meet a target retrieval quality, while top-k specifies the results returned to generation [22, 30]. Consequently, intermediate rankings are discarded even when further probing produces little change in retrieved evidence, and all queries remain bound to the same search budget regardless of their convergence behavior.

2.2 Retrieval-Grounded Generation

RAG conditions LLM generation on retrieved evidence by materializing the retrieval results into the input prompt. Given a query q and its ranked top-k passages $\{d_1, d_2, \ldots, d_k\}$, a RAG system instantiates a prompt template containing the task instruction and query, and appends the retrieved passages to the context in retrieval order [25, 30]. The resulting input can be represented as $\mathcal{P}(q) \parallel d_1 \parallel d_2 \parallel \cdots \parallel d_k$, where $\mathcal{P}(q)$ denotes the template instantiated with the query and $\parallel$ denotes sequence concatenation. The complete sequence is then submitted to the LLM. Top-k therefore determines not only how much retrieved evidence is exposed to the model, but also the length of the context processed during inference.

The cost of processing this retrieved context is incurred primarily during *prefill*. Before autoregressive decoding, the LLM processes all tokens in the augmented prompt and materializes their key and value states in the KV cache [40, 61]. Adding retrieved passages increases the input length and consequently the prefill computation and KV-cache footprint. During decoding, KV caching avoids recomputing states for previously processed tokens [29, 39, 59], but the initial computation required to materialize these states remains unavoidable. Hence, under the conventional static top-k execution, all selected passages are retrieved and incorporated before generation begins, and their associated prefill cost is paid regardless of their eventual utility to the generated response.

The sequential structure of the augmented prompt also determines whether previously computed states can be reused as the context evolves. Prefix caching retains KV states for an existing prompt prefix and reuses them when new tokens are appended [10, 32, 60]. For example, extending $\mathcal{P}(q)\|d_1$ to $\mathcal{P}(q)\|d_1\|d_2$ preserves the cached states of the original prefix, requiring prefill only for the appended context. In contrast, inserting, removing, or reordering earlier passages changes the token prefix and invalidates subsequent cached states. This append-only reuse property provides an mechanism for extending retrieved context without repeatedly prefilling evidence that has already been processed.

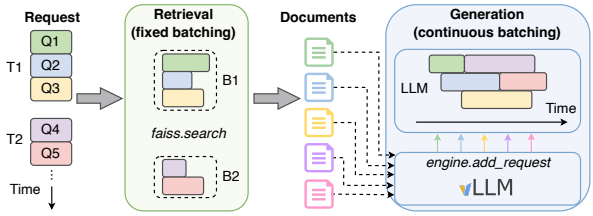


Figure 2: Heterogeneous batching in RAG serving, with fixed batching in retrieval and continuous batching in generation.

2.3 Cross-Stage RAG Execution

RAG serving typically integrates separately optimized retrieval and generation backends through frameworks such as FlashRAG [22]. A representative stack pairs Faiss [9] for vector retrieval with vLLM [29] for generation. While the serving layer coordinates dataflow between them, each backend retains its native execution and batching semantics. End-to-end RAG execution therefore spans two separately scheduled stages with distinct batching models.

Figure 2 illustrates the contrasting batching semantics of retrieval and generation. Faiss exposes call-level batched search, where each `faiss.search` invocation processes a fixed query set to exploit batch parallelism [9]. Batch formation is managed by the serving layer: in Figure 2, requests Q_1–Q_3 available at T_1 form search call B_1, while later arrivals Q_4 and Q_5 enter B_2. Once dispatched, the query set remains fixed until the synchronous call returns. In contrast, vLLM employs continuous batching, dynamically admitting retrieved requests through `engine.add_request` and retiring completed sequences during iterative LLM execution [29, 59]. Retrieval therefore operates on fixed call-level batches, whereas generation continuously updates its active request set.

This batching mismatch further governs how requests cross the retrieval–generation boundary. As illustrated by Q_2 in Figure 2, although it is an independent request, its retrieval is executed within B_1, and its results become visible to the serving layer only when the `search` call returns. Thus, even if queries in B_1 progress differently, their results remain constrained by the same call-level completion boundary. Once released, the retrieved passages are materialized into the prompt and submitted to vLLM, where Q_2 can be independently admitted through continuous batching. This creates an asymmetric cross-stage handoff in which coarse-grained retrieval completion gates access to fine-grained generation scheduling.

3 MOTIVATION

3.1 Rank-Sensitive Retrieval Convergence

While conventional RAG retrieval exhausts the configured *nprobe* before completion, it remains unclear how much each additional probe contributes to retrieval quality. To quantify this effect, we evaluate IVF under a series of static *nprobe* configurations and measure how retrieval quality changes as additional partitions are searched. Figure 3(a) shows consistent retrieval saturation across NQ, SQuAD, and HotpotQA. Retrieval coverage rises rapidly at small *nprobe*, reaching approximately 80% at *nprobe* =4, 6, and

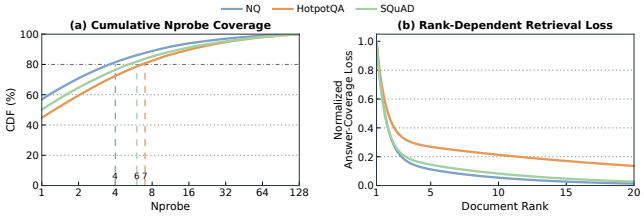


Figure 3: Search convergence and rank sensitivity in retrieval.

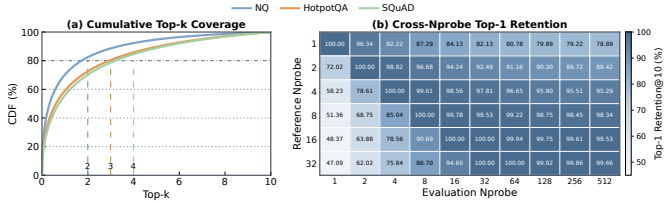


Figure 4: Evidence concentration and persistence in retrieval.

7, respectively, but improves only marginally as the search scope expands further. Additional probing thus yields rapidly diminishing quality gains while incurring increasing computational cost. This diminishing return suggests that retrieval completion should reflect search convergence rather than exhaustion of the search scope.

The remaining challenge is to identify such convergence online, since exact retrieval quality is unavailable at runtime. IVF provides such information through the evolving top-k ranking produced as successive partitions are probed. However, ranking changes are not equally informative, as leading-rank changes substantially affect retrieval quality while tail fluctuations have limited impact. To quantify rank sensitivity, we perturb retrieved documents at different ranking positions and measure the normalized answer-coverage loss. As shown in Figure 3(b), perturbations at the leading ranks incur substantially larger coverage loss, with the impact decreasing rapidly toward lower-ranked positions. The decay is sharper on NQ and SQuAD, whereas HotpotQA exhibits a broader rank-sensitive profile, consistent with its multi-hop setting where complementary evidence may span multiple retrieved documents. This result confirms that retrieval quality is strongly rank-sensitive, rather than uniformly determined by the entire top-k set.

These results distinguish *retrieval convergence* from complete *result-set stability*. A retrieval top-k may continue to fluctuate at lower ranks even after high-impact results have converged, without materially changing retrieval quality. Moreover, similar degrees of ranking variation can imply different search utility depending on where the changes occur. Changes among leading results signal meaningful uncertainty in retrieval quality, whereas tail fluctuations carry substantially less significance. Retrieval convergence is therefore better characterized by rank-sensitive result evolution than by uniform stabilization of the entire top-k set.

Observation 1. *Retrieval convergence can precede search-budget exhaustion, with high-rank evolution signaling search sufficiency.*

3.2 Early Evidence Persistence

Retrieval convergence determines how far the search needs to proceed, while generation efficiency further depends on how much retrieved evidence is processed. We first examine how retrieval evidence is distributed across ranking positions by varying top-k and measuring the cumulative coverage of the retrieved results. Figure 4(a) reports the resulting coverage across NQ, HotpotQA, and SQuAD, normalized against the top-10 retrieval result. The curves rise sharply at small k and then gradually saturate. Approximately 80% of the top-10 coverage is already captured by the top-2, top-3, and top-4 documents on NQ, HotpotQA, and SQuAD, respectively. The remaining documents therefore contribute progressively smaller coverage gains while increasing the context processed by the LLM. This concentration shows that a small prefix of the ranked results captures much of the evidence available to generation.

Evidence concentration alone, however, does not establish that these documents can be consumed before retrieval completes. Early generation requires the leading results observed at shallow search depths to remain reliable as more partitions are probed. We quantify this property through cross-*nprobe* retention. For each query, we take the top-1 document obtained at a reference *nprobe* and measure whether it remains within the top-10 results produced at a larger *nprobe*. Figure 4(b) reports the retention rate across reference and evaluation search depths. Results show strong persistence even at modest reference search depths. For example, the top-1 result at $nprobe = 4$ remains within the top-10 at $nprobe = 512$ for 95.29% of queries, while increasing the reference depth to $nprobe = 8$ raises retention to 98.34%. More generally, retention approaches 100% at moderate reference depths, even as the subsequent search scope expands by orders of magnitude. High retention suggests that deeper probing mainly refines ranking and introduces new candidates, while early top-ranked evidence remains stable.

Evidence concentration and persistence jointly show that useful context becomes available progressively as retrieval proceeds. High-ranked results capture substantial retrieval coverage at small context depths and remain reliable as deeper search continues to refine the ranking and discover new candidates. The retrieval state therefore evolves incrementally in both evidence coverage and ranking maturity, rather than becoming usable only when the final result set is established. This progressive evolution separates reliable evidence availability from retrieval completion, allowing generation to consume useful context while retrieval continues.

Observation 2. *Evidence readiness can precede retrieval completion, decoupling useful context availability from final ranking.*

3.3 Intra-Batch Retrieval Skew

Query-adaptive RAG exposes a fundamental mismatch between per-query progress and batch-level execution. Queries can reach retrieval convergence at different search depths, while useful evidence may become available before retrieval completes. These query-level opportunities, however, do not directly translate into earlier generation under conventional call-level retrieval batching. Requests within the same search invocation share a common completion boundary, causing generation admission to depend on the retrieval tail rather than individual query progress.

We quantify this mismatch by examining intra-batch variation in retrieval scan volume. Even under fixed retrieval, identical search depths do not imply identical per-query work. A fixed *nprobe* equalizes the number of IVF lists probed across queries, but variations in list cardinality result in different scan volumes. As shown in Figure 5(a), over the first 200 batches at 16 requests/s, the batch-wise

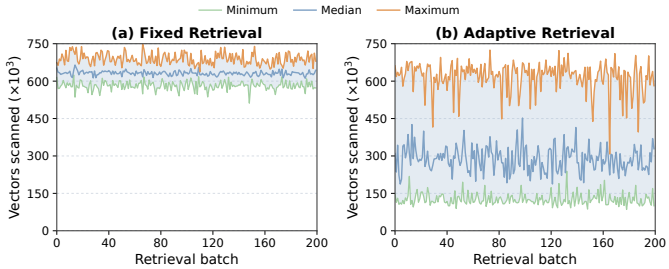


Figure 5: Intra-batch skew at 16 RPS and *nprobe* = 128.

minimum, median, and maximum scan volumes average 581.7K, 631.7K, and 689.1K vectors, respectively, yielding a mean min–max gap of 107.4K vectors. Query adaptation further amplifies this disparity, as query-dependent search depths compound the inherent variation in list cardinality. Under HarmoRAG's adaptive retrieval with a maximum *nprobe* of 128, Figure 5(b) shows that the mean minimum and median drop sharply to 126.2K and 287.5K vectors, while the mean maximum remains 616.3K. The resulting min–max gap grows to 490.1K vectors, 4.56× that of fixed retrieval. The reduction is therefore concentrated among early-terminating queries, while deeper-search queries continue to define the batch tail.

This amplified skew matters because per-query retrieval progress remains constrained by a batch-level release boundary. Adaptive retrieval can substantially reduce the work of queries that converge early, but under call-level batching their results remain unavailable until the slowest query completes. The resulting tail waiting prevents reductions in per-query search work from proportionally advancing generation admission. Increasing batch size improves retrieval throughput through greater query-level parallelism but exacerbates this waiting, whereas smaller batches reduce synchronization at the cost of batching efficiency. Conventional batching therefore creates a fundamental trade-off between retrieval efficiency and timely per-query handoff to generation.

Observation 3. *Query-adaptive retrieval creates asynchronous per-query progress, while batch-level synchronization prevents early progress from translating into downstream latency savings.*

4 DESIGN AND IMPLEMENTATION

In this section, we present HarmoRAG, a query-adaptive RAG serving system that exploits runtime retrieval progress to coordinate execution across the retrieval–generation pipeline. Rather than optimizing retrieval and generation in isolation, HarmoRAG adapts their execution jointly at the query level through three mechanisms: *rank-aware early termination* dynamically controls retrieval effort, *speculative incremental augmentation* progressively adapts generation context, and *cross-stage continuous batching* coordinates fine-grained execution across stages. Together, these mechanisms enable efficient end-to-end query-adaptive RAG serving.

4.1 Rank-Aware Early Termination

HarmoRAG dynamically allocates retrieval effort through *rank-aware early termination*. Unlike predictor-based approaches that estimate query difficulty or search depth using offline-trained models [6, 31, 55], HarmoRAG determines search sufficiency directly from ranking states observed during retrieval, eliminating predictor training. Terminus [36] also exploits online rank signals, but estimates the marginal utility of recent search steps from rank-weighted candidate insertions, making its termination decision sensitive to transient lulls in high-utility discoveries. HarmoRAG instead detects convergence at the ranking-state level through persistent bidirectional drift across successive top-D rankings, capturing candidate insertion, eviction, and rank displacement. By jointly modeling drift magnitude and temporal volatility, it terminates only after ranking stability persists, rather than relying on the short-term utility of recent search steps.

Rank-aware retrieval drift. HarmoRAG represents IVF retrieval as a sequence of ranking states to characterize search convergence. After probing s lists, it maintains $R_s^{(D)} = (d_{s,1}, d_{s,2}, \ldots, d_{s,D})$, where D is the monitored ranking depth and $d_{s,r}$ denotes the document at rank r. Each additional probe introduces new candidates and transforms the ranking from $R_{s-1}^{(D)}$ to $R_s^{(D)}$, exposing how retrieved evidence evolves with the search scope. Transitions at influential ranks indicate continued search utility, whereas changes confined to lower ranks suggest diminishing returns.

To translate these position-dependent ranking transitions into a quantitative convergence signal, HarmoRAG introduces *Weighted Rank Drift* (WRD), a rank-sensitive measure inspired by weighted ranking metrics [2, 27, 36, 51]. WRD weights ranking changes according to their positional significance rather than treating all positions in the monitored top-D uniformly. Specifically, each rank r is assigned a normalized positional weight:

$$w_r^{(D,\alpha)} = \frac{r^{-\alpha}}{\sum_{i=1}^{D} i^{-\alpha}}, \tag{1}$$

where α controls the decay of positional importance with rank. The weights satisfy $w_1 > w_2 > \cdots > w_D > 0$, emphasizing changes at influential positions over tail fluctuations. This weighting directly incorporates rank sensitivity into the convergence metric.

HarmoRAG decomposes the transition between consecutive states $R_{s-1}^{(D)}$ and $R_s^{(D)}$ into directional rank variations. The *forward drift* quantifies changes to documents in $R_{s-1}^{(D)}$ after further probing and is defined as:

$$C_{s-1\to s}^{(D)} = \sum_{d \in R_{s-1}^{(D)}} c_{s-1\to s}(d), \tag{2}$$

where

$$c_{s-1\to s}(d) = \begin{cases} w_{\pi_{s-1}(d)}, & \pi_s(d) = \perp, \\ \left| w_{\pi_s(d)} - w_{\pi_{s-1}(d)} \right|, & \pi_s(d) \neq \pi_{s-1}(d), \\ 0, & \pi_s(d) = \pi_{s-1}(d). \end{cases} \tag{3}$$

Here, $\pi_s(d)$ denotes the rank of document d in $R_s^{(D)}$, while $\perp$ indicates that it falls outside the monitored top-D. Documents that disappear incur their previous positional weight, rank displacement is penalized by the change in positional importance, and unchanged documents contribute zero drift. This formulation makes the transition cost sensitive to both extent and location of ranking changes, assigning greater drift to variations at influential positions.

Forward drift characterizes the displacement or disappearance of previously observed documents, but remains insensitive to candidates newly entering $R_s^{(D)}$. To account for both sides of the ranking transition, a complementary *backward drift* $C_{s\to s-1}^{(D)}$ is computed

by reversing the two ranking states, thereby accounting for newly introduced candidates under the same positional weighting. The resulting bidirectional drift is defined as:

$$\Delta_s = \mathrm{clip}_{[0,1]}\left(\frac{C^{(D)}_{s-1\to s} + C^{(D)}_{s\to s-1}}{2}\right). \tag{4}$$

The bidirectional formulation captures document insertion, removal, and rank displacement within a rank-sensitive measure. A large Δ_s indicates that probing continues to alter influential portions of the ranking, whereas a value approaching zero indicates diminishing changes to high-impact evidence. Consequently, Δ_s serves as an online measure of rank-sensitive search utility and provides the instantaneous signal required for convergence detection.

Temporal convergence estimation. While Δ_s quantifies a single rank-sensitive transition, an isolated low drift does not necessarily indicate convergence. The ranking may temporarily stabilize before a later IVF list perturbs the leading results, creating a transient plateau. HarmoRAG therefore examines the recent drift sequence $\Delta_{s-m+1}, \ldots, \Delta_s$ and aggregates the magnitude of ranking variation using an exponentially weighted moving average:

$$A_s = \frac{\sum_{j=0}^{m-1} \beta^j \Delta_{s-j}}{\sum_{j=0}^{m-1} \beta^j}, \tag{5}$$

where m defines the observation window and $0 \le \beta < 1$ controls temporal decay. The weighting emphasizes recent transitions while retaining historical drift within the window, allowing A_s to summarize recent ranking dynamics. A small A_s therefore indicates consistently limited variation over the observation window.

However, averaging may still mask intermittent ranking changes, making A_s alone insufficient for sustained convergence detection. Temporal fluctuation is therefore quantified as:

$$V_s = \frac{\sum_{j=0}^{m-2} \beta^j \left|\Delta_{s-j} - \Delta_{s-j-1}\right|}{\sum_{j=0}^{m-2} \beta^j}. \tag{6}$$

While A_s measures the magnitude of recent ranking variation, V_s quantifies its temporal fluctuation through changes between successive drift values. Reliable convergence therefore requires both low drift magnitude and low fluctuation.

Together, A_s and V_s define a retrieval uncertainty score:

$$\rho_s = A_s + \mu V_s, \tag{7}$$

where μ controls the contribution of temporal fluctuation. A low ρ_s indicates limited and stable ranking variation over the observation window. By combining both dimensions, ρ_s provides a more robust convergence signal than average drift, reducing false convergence from transient plateaus or oscillating trajectories.

Termination policy. HarmoRAG converts the uncertainty score into a robust termination decision by requiring persistent convergence. It maintains a stability counter h_s, which increments whenever $\rho_s < \epsilon$ and resets otherwise. Retrieval terminates only after this condition holds for P consecutive observations. This persistence requirement filters transient low-uncertainty intervals, preventing premature termination while allowing stable queries to stop before exhausting the full search budget.

If sustained convergence is not detected, probing continues until the maximum budget $N_{\max}$ is reached, providing a conservative fallback for queries with persistent ranking uncertainty. Table 1 summarizes the default parameters, which jointly govern rank sensitivity, temporal robustness, and termination conservativeness. As evaluated later in Figure 14 (§5.4), the selected operating point balances retrieval fidelity and search efficiency, as more conservative settings yield diminishing recall gains at higher latency. We set $D = 100$ and $\alpha = 1.0$ to monitor a sufficiently broad ranking prefix while emphasizing high-impact positions; $m = 6$, $\beta = 0.4$, and $\mu = 0.75$ capture recent ranking dynamics while remaining responsive to new transitions and fluctuations; and $\epsilon = 0.001$ with $P = 16$ imposes a conservative persistence requirement. $N_{\max} = 128$ bounds retrieval effort when convergence remains uncertain.

Table 1: Parameters for rank-aware early termination.

Parameter	Value	Description
D	100	WRD ranking depth
α	1.0	Rank decay factor
m	6	Drift history window
β	0.4	Temporal decay factor
μ	0.75	Fluctuation weight
ϵ	0.001	Convergence threshold
P	16	Stable observation count
$N_{\max}$	128	Maximum probed clusters

These controls transform a fixed IVF budget into a query-specific effective *nprobe*. Queries whose influential ranks stabilize quickly terminate with less exploration, whereas queries with persistent uncertainty continue toward $N_{\max}$. The maximum budget thus becomes a conservative fallback rather than a mandatory completion target, allowing retrieval effort to follow ranking convergence.

4.2 Speculative Incremental Augmentation

HarmoRAG also turns retrieval evidence from a fixed generation input into a progressively consumable context through *speculative incremental augmentation*. Conventional RAG constructs the generation context only after the complete top-k result is available, coupling generation to retrieval completion even when useful evidence emerges earlier. The evidence persistence characterized in §3.2 shows that high-ranked results can become available early and remain reliable as retrieval proceeds, providing an opportunity to relax this dependency. The key challenge is to exploit such intermediate evidence without committing to unreliable answers from incomplete contexts. HarmoRAG therefore preserves reliable evidence across generation rounds and incrementally extends the context as additional evidence becomes available.

Incremental context construction. Figure 6 illustrates the execution flow. Retrieval proceeds through successive search rounds, each refining the ranked document set as additional IVF lists are explored. Once the initial retrieval has been finished, HarmoRAG selects the current top-ranked unseen document and constructs the first generation context. As shown in Round 1, the prompt consists of the system instruction, question, and the initial document d_1 (Doc A). Rather than waiting for the complete top-k result, this compact context is immediately evaluated for generation while retrieval continues to refine the remaining evidence.

If the current evidence is insufficient, HarmoRAG preserves the existing context and appends the highest-ranked unseen document from the latest retrieval state. For example, Round 2 extends $\mathcal{P}(q)\|d_1$ to $\mathcal{P}(q)\|d_1\|d_2$, corresponding to Doc B in Figure 6. Subsequent rounds follow the same append-only structure until sufficient evidence is available or retrieval reaches its final state. Crucially,

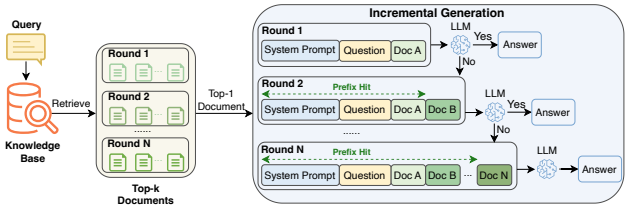


Figure 6: Incremental Generation of HarmoRAG.

previously incorporated documents are never reordered or reconstructed. The prompt prefix therefore remains unchanged across rounds, allowing the KV states of the system prompt, question, and accumulated evidence to be reused through prefix caching. As indicated by the *Prefix Hit* paths in Figure 6, each new round prefills only the newly appended evidence rather than recomputing the entire accumulated context. Incremental augmentation therefore increases evidence coverage while keeping the additional generation cost proportional to the newly introduced context.

Answerability gating. Progressive context construction enables earlier generation, but intermediate evidence may be insufficient for a reliable answer. HarmoRAG therefore treats each intermediate generation as speculative and commits the candidate answer only after an answerability gate is satisfied. At augmentation round j, document d_j is appended to the evidence to form the accumulated context x_j. The model then initiates a structured decoding request whose first token is constrained to N or A. An N decision signals insufficient evidence and terminates decoding while retrieval continues; an A decision indicates readiness to generate a candidate answer but does not authorize commitment.

For an A decision, HarmoRAG computes an answerability margin from the two tokens' log probabilities at the first decoding position:

$$\Delta P_j = \log P(A \mid x_j) - \log P(N \mid x_j). \tag{8}$$

Within the constrained A/N decision space, ΔP_j measures the model's preference for answering over requesting more evidence. Decoding proceeds to $A < answer >$ only when $\Delta P_j \geq \gamma$; we set $\gamma = 16$ by default, as evaluated in §5.4. Otherwise, decoding stops at the decision token, avoiding unnecessary autoregressive decoding.

Passing the confidence gate permits candidate generation, but final commitment additionally requires structural validation: the answer must be complete, nonempty, non-truncated and follow the prescribed $A < answer >$ format. A candidate failing any of the two checks—confidence and structural validity—is withheld, and retrieval continues to provide evidence for the next augmentation round. These checks ensure that speculative generation does not commit answers from insufficient intermediate contexts.

Progressive execution policy. HarmoRAG determines when to expose the first retrieval result and how to advance augmentation rounds. Starting too early risks operating on an unstable leading document, whereas waiting for full retrieval forfeits the latency opportunity identified in §3.2. Incremental generation therefore begins at $nprobe = 32$, where the leading retrieval result reaches sufficient empirical stability (Figure 4), with its top-ranked document serving as the initial evidence. Retrieval then continues in parallel with generation, progressively refining the candidate ranking.

During active retrieval, a failed answerability check preserves the accumulated context x_j and appends the highest-ranked unseen document from the latest top-D results, yielding $x_{j+1} = x_j \| d_{j+1}$. Previously incorporated documents are never removed or reordered, even if their relative ranks later change. Each round therefore adds exactly one new document while preserving the reusable prefix of x_j, progressively filling the top-k context budget with the strongest available evidence. If an intermediate round passes the answerability gate, its answer is committed and further retrieval evidence is unnecessary. Otherwise, once retrieval converges, HarmoRAG freezes the final top-D ranking, appends the remaining unseen documents in rank order until the top-k budget is filled, and performs a single final generation over the consolidated context. This policy exploits persistent evidence as soon as it becomes useful while retaining a full-context fallback when early evidence is insufficient.

Overall, *speculative incremental augmentation* converts retrieved context from a fixed input assembled after search completion into a progressively available execution state. Persistent evidence enables early context construction, append-only augmentation preserves prefix reuse across rounds, and answerability gating controls when speculative generation can safely become final. This design allows generation to exploit evidence as it becomes available while bounding redundant prefill and retaining a conservative final-generation path for queries that require deeper context.

4.3 Cross-Stage Continuous Batching

HarmoRAG further extends query-adaptive execution across the retrieval–generation boundary through *cross-stage continuous batching*. Conventional batch-level handoffs obscure the per-query progress skew described in §3.3, preventing generation from consuming already available evidence. Existing systems only partially relax this barrier. HedraRAG [14] trades stricter synchronization for speculative execution that may trigger regeneration, while VectorLiteRAG [26] removes intra-batch blocking but preserves cross-batch admission barriers. HarmoRAG instead exposes each query's progress independently across the stage boundary by combining resumable retrieval, dynamic batch membership, and bidirectional control between the retrieval and generation schedulers.

Resumable retrieval execution. A conventional Faiss `search` invocation fixes its batch membership and returns results only after all queries complete. Directly releasing an early-converged query is therefore difficult because its execution state is embedded within an in-flight batch. HarmoRAG removes this restriction at the inverted-list scanning stage while retaining batching for query encoding and coarse assignment. Specifically, each query q_i is represented by a persistent retrieval state:

$$S_i^r = (p_i, H_i, R_i, z_i^r),$$

where p_i identifies the next inverted list to probe, H_i stores the accumulated top-D candidates, R_i maintains the ranking history for convergence detection, with the top-D ordering from the first list scan ($nprobe = 1$) serving as the reference ranking, and $z_i^r \in \{\text{Accept}, \text{Continue}, \text{Finish}\}$ records the retrieval decision at each scheduling boundary. Accept indicates WRD-confirmed ranking stability, whereas Finish indicates exhaustion of the *nprobe* budget without earlier acceptance. Both terminate retrieval, release CPU capacity, and freeze the candidate heap for downstream generation. Otherwise, Continue preserves the query state and schedules the next inverted-list scan. Using Faiss's native

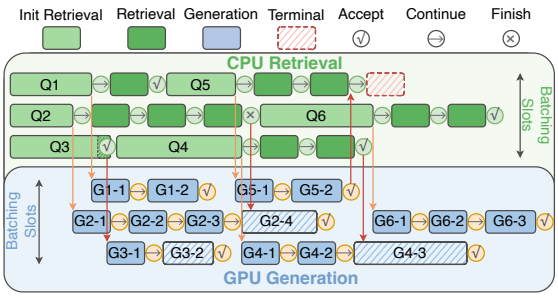


Figure 7: Cross-stage continuous batching in HarmoRAG.

`get_InvertedListScanner` interface, each scheduling quantum advances a query by one list and updates S_i^r. Retrieval can therefore pause and resume at list boundaries without repeating coarse assignment or rescanning explored lists.

Retrieval-side continuous batching. The upper half of Figure 7 shows how retrieval batch membership evolves dynamically. The scheduler maintains a bounded active set and a global waiting queue. Queries Q1–Q3 arrive together and occupy the available batching slots. After batched encoding and coarse assignment, each query advances independently through its probe sequence while preserving progress in persistent retrieval state.

At each inverted-list boundary, queries marked Continue retain their scan state for subsequent execution, while those entering Accept or Finish retire from the active set even if generation continues on the GPU. Freed slots are reassigned to waiting queries without waiting for the remaining active queries to finish retrieval. In Figure 7, Q4 is admitted as soon as Q3 retires, while Q1 and Q2 continue scanning. This per-query admission and retirement removes the batch-tail barrier of fixed-batch Faiss search.

Cross-stage coordination. To translate retrieval-side continuous batching into end-to-end gains, HarmoRAG coordinates the retriever and generator through a shared evidence queue. After each retrieval quantum, the retrieval worker publishes the updated per-query retrieval state S_i^r to the cross-stage coordinator and resumes from this state in the next quantum. The coordinator continuously tracks the latest retrieval state for each query. Updates arriving during an in-flight generation round refresh this state without involving the generator; newer rankings supersede earlier ones, and the latest H_i determines evidence selection in subsequent rounds.

A query becomes eligible for generation once its initial-evidence checkpoint is reached. The coordinator selects the highest-ranked document in H_i that has not yet been consumed, appends it to the accumulated context, and updates the query-local generation state:

$$S_i^g = (C_i, A_i, z_i^g),$$

where C_i stores the accumulated append-only context, A_i records the current answer candidate, and $z_i^g \in \{\text{Accept}, \text{Continue}\}$ denotes the generation decision. Accept commits the current answer, whereas Continue requests additional evidence.

The lower half of Figure 7 shows how per-query retrieval progress drives GPU execution. Q1–Q3 enter generation independently once their initial evidence becomes available, while retrieval continues concurrently. Each incremental generation round appends one newly selected document to the accumulated context C_i. If a round returns Continue, the coordinator first incorporates all available retrieval updates and then selects the highest-ranked unseen document from the latest H_i. Retrieval advances at inverted-list boundaries, whereas LLM server schedules generation at its native iteration granularity. The two schedulers thus progress asynchronously while preserving each query's evidence dependencies.

Cross-stage coordination also supports completion driven by either retrieval or generation. When retrieval converge before init retrieval completes as Q3, the coordinator terminates retrieval and initiates generation from the highest-ranked evidence, consolidating the remaining unseen documents only when a final generation is required. If retrieval instead converges or exhausts its search budget after init retrieval but before an answer is accepted, the coordinator freezes the final ranking, appends all remaining unseen documents within the context budget, and submits the consolidated context to the LLM server for one final generation, as illustrated by G2-4 and G4-3. Conversely, if an in-flight generation round returns Accept while retrieval is still active, the answer is committed and the coordinator sends a Terminal event to cancel the remaining scans at the next inverted-list boundary. The query then retires from retrieval, releasing CPU capacity to waiting queries. This bidirectional feedback allows either stage to terminate the remaining work once further retrieval or generation becomes unnecessary.

Cross-stage coordination gives each query a unified completion condition across retrieval and generation. Progress in either stage can terminate work that is no longer useful in the other, while unresolved queries continue to consume newly available evidence without imposing cross-query synchronization. This feedback-driven execution turns independently scheduled retrieval and generation into a query-progress-aware pipeline, allowing resources to follow useful work rather than stage boundaries or batch completion.

5 EVALUATION

5.1 Experimental Setup

Hardware. All experiments are conducted on a server equipped with an AMD EPYC 7742 64-core processor for vector retrieval and eight NVIDIA A100 40 GB NVLink GPUs for LLM generation, with SSD/NVMe storage supporting large-scale data access.

Models. We use Llama3–8B [11] by default and evaluate Qwen3-32B [54] and Llama3-70B [11] to assess generality across model scales. All use instruction-tuned variants for RAG serving.

Corpus and Index. We use a Wikipedia corpus of approximately 21M passages [7] as the knowledge base for RAG evaluation. Each passage is encoded into a 1024-dimensional dense vector using E5-large [48]. To evaluate retrieval across different dataset scales and dimensionalities, we additionally use three standard ANNS benchmarks: SIFT100M [18], DEEP100M [4], and GIST1M [17]. We build IVF4096 indexes for all retrieval corpora.

Datasets. We evaluate HarmoRAG on both end-to-end RAG serving and ANNS retrieval. For end-to-end evaluation, we use three representative QA datasets: SQuAD [41] (10,570 queries), Natural Questions [28] (79,168 queries), and HotpotQA [57] (90,447 queries). They cover passage comprehension, single-hop factoid QA and multi-hop reasoning with evidence aggregation. For ANNS evaluation, we randomly sample 10,000 queries from SIFT100M and DEEP100M, and 1,000 from GIST1M.

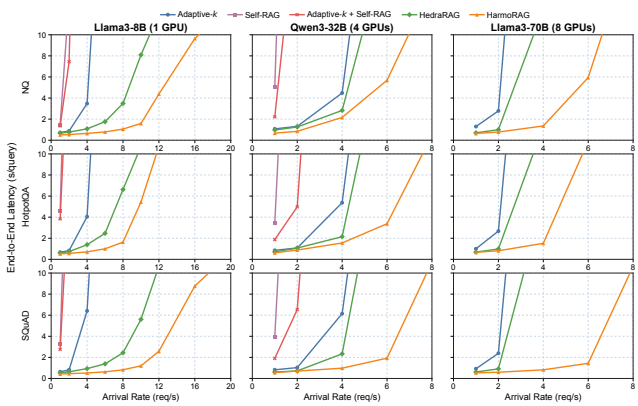


Figure 8: End-to-end latency comparison.

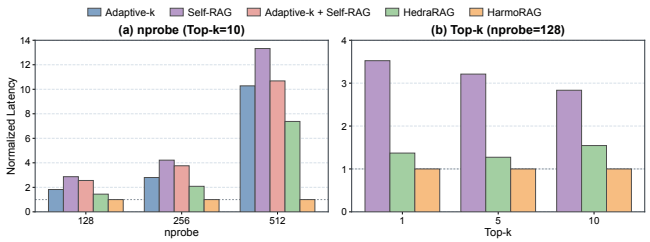


Figure 9: Latency under different retrieval configurations.

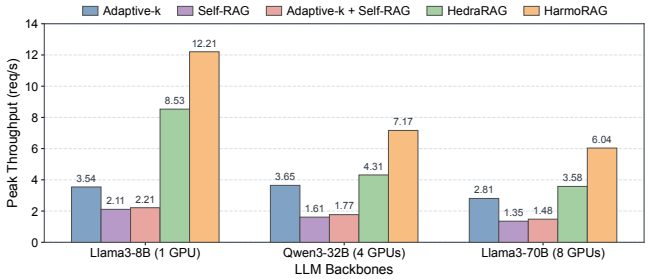


Figure 10: Peak throughput across LLM scales.

Baselines. We compare HarmoRAG against baselines covering adaptive retrieval, context selection, and end-to-end RAG serving. DARTH [6] provides recall-driven early termination for ANNS, while LAET [31] uses a learned model to adapt search termination. Terminus [36] triggers early termination from a short window of low-utility insertion-rank observations. Adaptive-k [44] dynamically selects the number of passages supplied to generation. Notably, recent query-adaptive systems such as METIS [42] rely on unreleased prediction models and therefore cannot be faithfully reproduced as experimental baselines. We thus include Self-RAG [1] and HedraRAG [14] as end-to-end baselines for adaptive retrieval–generation control and high-performance RAG serving. Self-RAG + Adaptive-k combines adaptive RAG execution with query-specific context selection. VectorLiteRAG [26] serves as a targeted baseline for early departure from fixed retrieval batches.

Metrics. We evaluate both retrieval and answer quality. For ANNS, Recall@k measures the fraction of exact top-k neighbors recovered by approximate search. For end-to-end RAG, we report F1 and accuracy to measure answer overlap and containment, respectively.

5.2 Overall Improvement

End-to-End Performance. We first evaluate end-to-end RAG serving across three QA datasets and three LLM backbones under increasing request arrival rates, with $nprobe = 128$. We set the maximum top-k to 10 based on Adaptive-k's average selection of 7.8 passages per query, ensuring a common context budget across methods. As shown in Figure 8, HarmoRAG consistently achieves lower end-to-end latency across all nine dataset–backbone combinations. The advantage is pronounced even at low load: at 1 req/s, HarmoRAG achieves 2.84–8.69× latency speedups over Self-RAG and 2.70–7.30× over Adaptive-k + Self-RAG on the 8B and 32B backbones. Adaptive context selection improves Self-RAG, yet the combined baseline remains substantially slower than HarmoRAG, showing that query-level adaptation alone does not directly translate into end-to-end serving gains.

These gains persist across Adaptive-k and HedraRAG at all model scales. At 2 req/s, HarmoRAG achieves mean latency speedups of 1.63×, 1.42×, and 3.64× over Adaptive-k with the 8B, 32B, and 70B backbones, respectively, and 1.37×, 1.24×, and 1.35× over HedraRAG. The advantage becomes more pronounced as load increases. With Llama3-8B at 4 req/s, HarmoRAG maintains mean latency of 0.52–0.71 seconds, corresponding to 5.37–12.32× speedups over Adaptive-k. At 8 req/s, HarmoRAG maintains mean latency 2.96–4.06× speedups over HedraRAG. These results show that HarmoRAG sustains low latency across a broader load range rather than only under lightly loaded execution.

These latency reductions translate into higher SLO-constrained serving capacity, defined as the highest tested arrival rate satisfying a 10-second mean-latency SLO [14]. HarmoRAG consistently supports higher request rates than Adaptive-k and HedraRAG across model scales, with gains of up to 4.00× and 3.00×, respectively. Self-RAG-based baselines exhibit a much narrower load range; with Llama3-70B, both violate the SLO even at the lowest tested arrival rate. Overall, HarmoRAG sustains lower latency under increasing load and supports a wider SLO-compliant range by reducing low-yield work and cross-stage queueing.

Performance Across Retrieval Parameters. We further examine how $nprobe$ and top-k affect end-to-end latency. As shown in Figure 9(a), HarmoRAG's latency advantage grows with $nprobe$. As $nprobe$ increases from 128 to 512, baseline latency rises from 1.44–2.87× to 7.38–13.32× that of HarmoRAG. The widening gap reflects the cost of exhausting larger search budgets, whereas HarmoRAG terminates retrieval once the ranking has sufficiently converged.

Figure 9(b) evaluates the effect of top-k with $nprobe = 128$. We include Self-RAG, HedraRAG, and HarmoRAG, as Adaptive-k and Adaptive-k + Self-RAG select 7.8 passages per query and are unsuitable for controlled comparison at small top-k. Increasing top-k enlarges the generation context and raises cost for all three systems. HarmoRAG nevertheless maintains the lowest latency, reducing end-to-end latency by 64.7%–71.6% over Self-RAG and 21.4%–35.3% over HedraRAG. These results show that HarmoRAG remains effective across retrieval settings by limiting search to converged rankings and generation to query-relevant evidence.

Performance on Peak Throughput. Figure 10 compares the peak throughput of the five systems across three model scales. To measure serving capacity independently of the 10-second latency

Table 2: Mean probing cost on QA and ANN workloads.

Dataset	Recall @100	Mean *nprobe*				Reduction (%)
		LAET	DARTH	Terminus	HarmoRAG	
NQ	0.93	–	–	128.00	**57.42**	55.14
HotpotQA	0.89	–	–	128.00	**74.27**	41.98
SQuAD	0.90	–	–	128.00	**63.27**	50.57
SIFT100M	0.90	20.65	18.87	16.40	**15.59**	4.94
	0.95	35.30	29.78	26.51	**25.42**	4.11
	0.99	83.16	65.73	128.00	**58.89**	10.41
DEEP100M	0.90	17.18	14.54	12.14	**11.76**	3.13
	0.95	30.54	23.65	20.20	**19.43**	3.81
	0.99	81.18	54.90	128.00	**48.03**	12.51
GIST1M	0.90	103.60	101.19	128.00	**89.44**	11.61
	0.95	165.00	156.01	256.00	**137.14**	12.10
	0.99	355.80	312.78	512.00	**269.66**	13.79

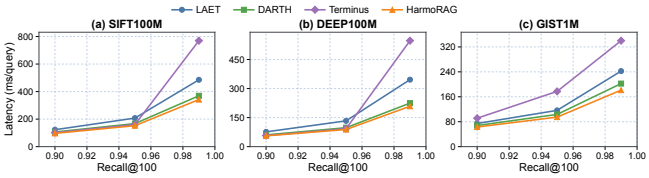


Figure 11: Efficiency of query-adaptive retrieval methods.

SLO, we increase the offered load until the request-completion rate plateaus and report the maximum observed throughput. HarmoRAG achieves the highest throughput in all settings, reaching 12.21, 7.17, and 6.04 requests/s with the 8B, 32B, and 70B models, respectively. These rates are 1.43×, 1.66×, and 1.69× those of HedraRAG, the strongest baseline in each setting. The gains result from query-adaptive retrieval, incremental context processing, and cross-stage coordination, which reduce per-request work while sustaining retrieval–generation concurrency under saturation.

5.3 Intra- and Cross-Stage Performance

Performance on Retrieval. We quantify the retrieval cost of HarmoRAG on three QA workloads. As reported in Table 2, HarmoRAG achieves Recall@100 values of 0.93, 0.89, and 0.90 on NQ, HotpotQA, and SQuAD while probing only 57.42, 74.27, and 63.27 IVF lists on average, respectively. When calibrated to these recall targets, Terminus exhausts the full *nprobe* = 128 budget because transient low-utility plateaus leave its short-window rule without a recall-compliant stopping point. HarmoRAG distinguishes these plateaus from sustained convergence through persistence-aware aggregation, reducing probing effort by 41.98%–55.14%. To assess whether this reduction generalizes beyond QA workloads, we compare HarmoRAG with LAET, DARTH and Terminus under matched Recall@100 targets of 0.90, 0.95, and 0.99. Across these benchmarks, HarmoRAG requires 3.13%–13.79% fewer probes than the most probe-efficient baseline at the same recall target, while Terminus exhausts the full probe budget at 0.99 recall on SIFT100M and DEEP100M and at all evaluated recall targets on GIST1M.

Figure 11 further shows that the reduction in probing effort translates into lower online retrieval latency while satisfying every prescribed Recall@100 target. Across all dataset–recall configurations, HarmoRAG achieves the lowest latency, improving upon the strongest baseline at each target by 2.42%–10.40%. On the two 100-million-vector datasets, it reduces latency by 21.5%–39.7% relative to LAET and by 7.2%–9.5% relative to DARTH. The advantage becomes more pronounced when Terminus exhausts its search budget—at Recall@100=0.99 on SIFT100M and DEEP100M and at every target on GIST1M. In these settings, HarmoRAG lowers latency by 30.9%–61.9% relative to Terminus, while retaining a 7.1%–10.4% advantage over DARTH, the strongest alternative on GIST1M. Collectively, these results demonstrate that query-specific convergence detection enables HarmoRAG to sustain a superior latency–recall trade-off across corpus scales and recall requirements.

Table 3: Offline preprocessing time on ANNS benchmarks.

System	SIFT100M	DEEP100M	GIST1M
DARTH	112.03 min	85.25 min	15.30 min
LAET	59.52 min	17.57 min	4.00 min
Terminus	6.81 min	7.18 min	1.99 min
HarmoRAG	6.84 min	7.21 min	2.05 min

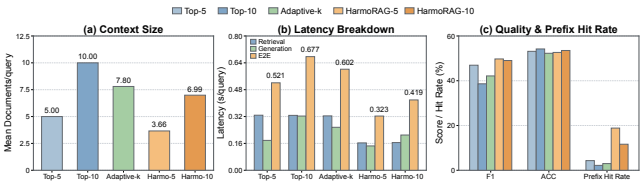


Figure 12: Generation performance with *nprobe* = 128.

Beyond online retrieval, the four systems incur fundamentally different offline preparation costs. DARTH constructs fine-grained search traces and supervised features to train and calibrate a termination predictor, while LAET performs feature extraction and model training to estimate query-specific search budgets. Terminus and HarmoRAG, in contrast, avoid supervised predictor training: Terminus calibrates utility-based termination parameters from progressive search traces, whereas HarmoRAG replays recorded ranking histories to select WRD parameters on held-out validation data. Under the same paired measurement protocol, Table 3 shows that Terminus requires 6.81, 7.18, and 1.99 minutes on SIFT100M, DEEP100M, and GIST1M, respectively, while HarmoRAG requires 6.84, 7.21, and 2.05 minutes. Their offline costs are therefore comparable, with HarmoRAG reducing preparation time by 86.6%–93.9% relative to DARTH and by 48.6%–88.5% relative to LAET. These results establish trace-based parameter calibration as a lightweight alternative to predictor-based training for query-adaptive retrieval.

Performance on Generation. We next examine the generation-side benefits of HarmoRAG through adaptive context construction. Top-5 and Top-10 denote conventional full-context execution with top-k fixed at 5 and 10, respectively. HarmoRAG-5 and HarmoRAG-10 use the same context budgets but progressively augment the generation context and stop once sufficient evidence has been accumulated. The suffix in HarmoRAG-k denotes the maximum context budget, not the number of documents consumed. We also include Adaptive-k as a representative adaptive context-selection baseline.

Figure 12 compares effective context size, stage-wise latency, and answer quality. HarmoRAG reduces the mean context size from 5.00 to 3.66 documents under $k = 5$ and from 10.00 to 6.99 under $k = 10$, corresponding to reductions of 26.8% and 30.1%, respectively. It also processes fewer documents than Adaptive-k, which selects 7.8 passages per query on average. The reduced context lowers generation latency by 18.6% under $k = 5$ and 35.0% under $k = 10$. These

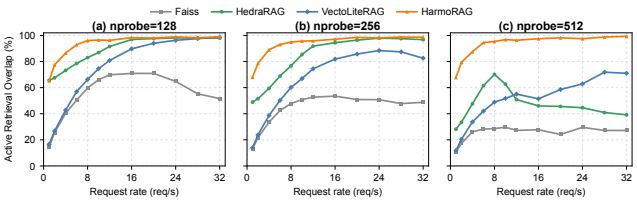


Figure 13: Active retrieval overlap (top-k = 1).

generation-side gains translate into end-to-end latency reductions of 38.0% and 38.1%, respectively. Compared with Adaptive-k, HarmoRAG reduces generation latency by 18.0%–43.5% and end-to-end latency by 30.4%–46.4%, demonstrating the benefit of progressively consuming only the evidence needed for generation.

These efficiency gains do not come at the expense of answer quality. HarmoRAG-5 and HarmoRAG-10 achieve higher F1 with comparable ACC; under $k = 10$, HarmoRAG-10 reaches 49.39% F1, compared with 38.91% for Top-10. This gap suggests that indiscriminately adding retrieved documents can introduce marginally relevant or conflicting evidence even when retrieval coverage is unchanged. Moreover, prefix-cache hit rates increase from 4.31% to 18.85% under $k = 5$ and from 2.19% to 11.64% under $k = 10$, representing 4.37× and 5.32× improvements. By incrementally admitting evidence, HarmoRAG converts a fixed context budget into a compact, query-dependent context, reducing generation latency while preserving, and in some cases improving, answer quality.

Performance on Cross-Stage Execution. We quantify cross-stage concurrency using *active retrieval overlap*, defined as $H_{\text{active}} = \frac{|R \cap G|}{|R|}$, where R and G denote the unions of physical retrieval-service and useful generation-service intervals, respectively, excluding queueing, loading, idle polling, and discarded speculative generation. Accordingly, H_{active} can characterize temporal concurrency.

Figure 13 compares the four systems under top-$k = 1$ across varying request rates and retrieval budgets. The batch-oriented Faiss baseline provides limited cross-stage concurrency, with its overlap plateauing or declining as retrieval becomes more expensive. VectoLiteRAG generally benefits from increased cross-request concurrency at higher loads, but its effectiveness diminishes with larger retrieval budgets. HedraRAG achieves stronger overlap at moderate budgets, yet becomes unstable at $nprobe = 512$ as retrieval saturation emerges. In contrast, HarmoRAG sustains consistently high overlap across all evaluated settings, averaging 91.9%, 92.2%, and 92.4% at $nprobe = 128$, 256, and 512, respectively. Across all matched configurations, HarmoRAG achieves 92.2% active retrieval overlap, compared with 71.4% for HedraRAG, 60.4% for VectoLiteRAG, and 40.5% for Faiss. These results show that HarmoRAG consistently converts fine-grained retrieval progress into useful retrieval–generation concurrency, remaining robust as both offered load and retrieval cost increase.

5.4 Ablation Studies

Analysis of Weighted Rank Drift. We analyze WRD from two aspects: the contribution of its drift statistics and the sensitivity to key parameters. As shown in Figure 14(a), raw drift captures instantaneous ranking changes but cannot distinguish sustained convergence from transient fluctuations. Average drift (A_s) captures the overall convergence trend, while volatility (V_s) captures

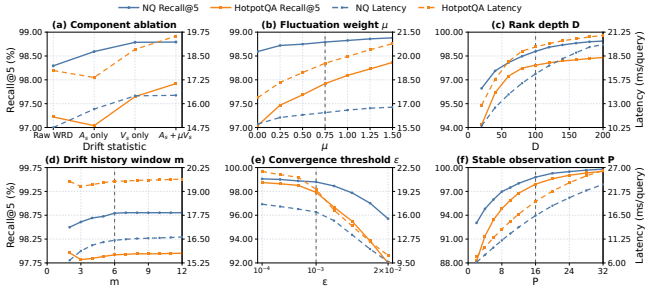


Figure 14: Sensitivity and ablation analysis of weighted rank drift on Recall@5 and retrieval latency.

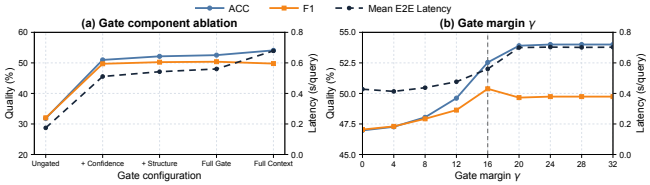


Figure 15: Answerability gate ablation and margin sensitivity.

short-term instability that may otherwise cause premature termination. Combining both yields Recall@5 of 98.79% on NQ and 97.92% on HotpotQA with controlled retrieval latency, confirming their complementary roles in convergence estimation.

We next vary ranking depth D, history window m, fluctuation weight μ, convergence threshold ϵ, and stable observation count P. These parameters control the trade-off between termination aggressiveness and retrieval fidelity. Increasing D from 100 to 200 raises Recall@5 on NQ from 98.79% to 99.44%, but increases latency from 16.44 to 19.81 ms. Similarly, increasing P from 16 to 32 improves Recall@5 on HotpotQA from 97.92% to 99.53%, while increasing latency from 19.54 to 26.36 ms. The diminishing recall gains indicate limited benefit from increasingly conservative termination. We therefore use $D = 100$, $m = 6$, $\mu = 0.75$, $\epsilon = 0.001$, and $P = 16$ by default, which consistently balances retrieval accuracy and latency. WRD incurs approximately 2μs per ranking update, accounting for only 0.7%–0.9% of overall retrieval latency.

Analysis of Answerability Gate. We evaluate the answerability gate from two aspects: the contribution of its components and the sensitivity to confidence margin γ. The gate determines whether a context provides sufficient evidence to commit an early answer. Figure 15(a) introduces verification components in order. The confidence check yields the largest improvement over ungated acceptance, raising ACC from 31.84% to 50.98% and F1 from 32.01% to 49.71% by rejecting low-confidence decisions before answer-body decoding. Structural validation excludes malformed candidates, bringing Full Gate to 52.54% ACC and 50.38% F1. Compared with Full Context (54.10% ACC and 49.77% F1), Full Gate preserves comparable answer quality while reducing latency by 17.5%.

We vary the confidence margin γ, which controls the conservativeness of early-answer acceptance. Figure 15(b) shows that smaller margins reduce latency through aggressive acceptance but increase the risk of insufficient evidence. Increasing γ from 0 to 16 improves ACC from 46.97% to 52.54% and F1 from 47.04% to 50.38%, while increasing latency from 0.428 to 0.560 s/query. Larger margins yield diminishing quality gains at higher latency. We use $\gamma = 16$ by default, which provides a quality–latency operating point.

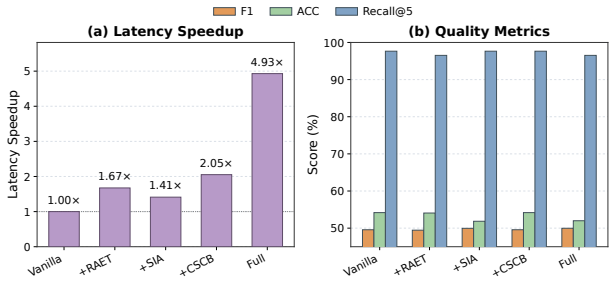


Figure 16: Component ablation of HarmoRAG.

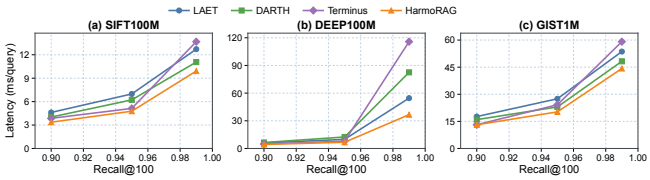


Figure 17: Query-adaptive retrieval on HNSW.

Component Ablation. We evaluate the contributions of three complementary optimizations in HarmoRAG under top-$k = 5$ and $nprobe = 128$: rank-aware early termination (RAET), speculative incremental augmentation (SIA), and cross-stage continuous batching (CSCB). As shown in Figure 16, the three mechanisms target distinct sources of inefficiency along the RAG execution path. RAET reduces low-yield retrieval work and delivers a 1.67× speedup; SIA limits unnecessary context processing and provides a 1.41× speedup; CSCB exposes cross-stage concurrency and further improves performance to 2.05×. When combined, these mechanisms achieve a 4.93× end-to-end latency speedup over vanilla RAG, showing that retrieval adaptation, progressive context construction, and cross-stage execution are complementary rather than substitutable.

These gains are achieved with limited quality impact. The full HarmoRAG configuration retains 96.52% Recall@5, 1.13% below the 97.65% achieved by vanilla RAG, while F1 improves from 49.57% to 49.97%. ACC decreases by 2.19%, from 54.18% to 51.99%, because SIA may construct an evidence sequence that differs from the complete retrieved context. Overall, the ablation confirms that HarmoRAG derives most of its efficiency gains from reducing low-yield work and exposing otherwise hidden cross-stage concurrency, while maintaining broadly stable retrieval and answer quality.

Extensions to HNSW. To examine whether query-adaptive execution generalizes beyond partition-based indexes, we instantiate HarmoRAG on HNSW with $M = 32$ and $efConstruction = 500$. Because HNSW traversal follows query-dependent graph topology rather than a fixed sequence of partitions, search progress is measured by cumulative distance computations, with *min_dists* preventing premature termination and *efSearch* bounding the exploration horizon. After the minimum-search requirement is satisfied, HarmoRAG terminates traversal only when the top-k ranking exhibits persistent convergence. Figure 17 compares all methods at matched Recall@100 on the same graph. Across SIFT100M, DEEP100M, and GIST1M, HarmoRAG reduces mean per-query latency by 26.9%, 24.7%, and 20.3% relative to LAET, DARTH, and Terminus, respectively. HarmoRAG can generalizes persistent rank convergence to prune low-yield expansions across indexes.

6 RELATED WORK

RAG Serving. RAG serving systems optimize the retrieval–generation pipeline through computation reuse, retrieval acceleration, and cross-stage coordination. RAGCache [21] exploits retrieval locality to reuse intermediate inference states, while VectorLiteRAG [26] improves index management and vector-search execution for resource-efficient retrieval. Beyond individual stages, HedraRAG [14] coordinates heterogeneous CPU–GPU resources to overlap retrieval and generation, while METIS [42] uses an auxiliary LLM to select a fixed context size and synthesis strategy, and RAGO [20] optimizes workload-level task placement, resource allocation, and batching. These systems substantially improve RAG execution, but generally treat retrieval progress and evidence consumption as stage-level decisions. HarmoRAG instead jointly adapt search, context construction, and cross-stage execution, extending stage-level optimization to continuous query-adaptive execution.

ANNS Optimization. ANNS systems improve retrieval efficiency through index design, system acceleration, and adaptive search. DiskANN [16], FreshDiskANN [43], and SPFresh [53] optimize index organization, storage-aware search, and dynamic maintenance for large-scale vector retrieval, while RAIRS [56] reduces redundant computation in IVF execution. A complementary line of work adapts search effort to individual queries. LAET [31] use learned models to predict early termination, while DARTH [6] determines stopping decisions under target recall constraints. Terminus [36] terminates retrieval after a short window of low-utility insertion-rank updates. HarmoRAG instead treats sustained convergence of the entire ranking as a runtime signal and propagates it across the RAG pipeline to drive query-adaptive execution.

Context Optimization. Context optimization reduces LLM inference cost through KV-cache reuse and efficient long-context processing. Prefix caching reuses KV states for shared prompt prefixes, with systems such as LMCache [35] and SGLang [60] providing efficient cache management. For partially overlapping RAG contexts, CacheBlend [58] enables selective KV reuse with partial recomputation. RetrievalAttention [34] and InfLLM [52] further reduce the computation and memory cost of long-context inference. These approaches primarily optimize the processing of an available context, whereas RAG evidence itself may emerge progressively during retrieval. HarmoRAG instead incrementally augments context with persistent evidence, enabling generation before retrieval completes while preserving prefix reuse across augmentation rounds.

7 CONCLUSION

We present HarmoRAG, a query-adaptive RAG serving system that uses runtime retrieval progress to coordinate execution across the retrieval–generation pipeline. HarmoRAG combines rank-aware early termination, speculative incremental augmentation, and cross-stage continuous batching to adapt search effort, evidence consumption, and pipeline execution to individual queries. Evaluations across diverse RAG workloads show that HarmoRAG substantially reduces end-to-end latency and improves throughput while preserving retrieval and answer quality. These results demonstrate that evolving retrieval progress can serve as a unifying runtime signal, enabling RAG systems to move beyond one-shot adaptation toward continuous, query-level execution control.

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, Vol. 2024. 9112–9141.

[2] Abolfazl Asudeh, HV Jagadish, Gerome Miklau, and Julia Stoyanovich. 2018. On obtaining stable rankings. *Proceedings of the VLDB Endowment* 12, 3 (2018), 237–250.

[3] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.

[4] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2055–2063.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[6] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. 2025. Darth: Declarative recall through early termination for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–26.

[7] Danqi Chen, Adam Fisch, Jason Weston, and Antoine Bordes. 2017. Reading wikipedia to answer open-domain questions. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1870–1879.

[8] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. 2022. Knowledge neurons in pretrained transformers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8493–8502.

[9] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The faiss library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361.

[10] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems* 6 (2024), 325–338.

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[12] Hao Guo and Youyou Lu. 2026. Achieving low-latency graph-based vector search via aligning best-first search algorithm with SSD. *ACM Transactions on Storage* 22, 2 (2026), 1–30.

[13] Shuang Hao, Xinxin Li, and Wei Zhang. 2026. Recall-Aware Early Termination in Approximate Nearest Neighbor Search. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 1578–1589.

[14] Zhengding Hu, Vibha Murthy, Zaifeng Pan, Wanlu Li, Xiaoyi Fang, Yufei Ding, and Yuke Wang. 2025. Hedrarag: Co-optimizing generation and retrieval for heterogeneous rag workflows. In *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles*. 623–638.

[15] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*. 874–880.

[16] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishaswamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems* 32 (2019).

[17] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2011. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2011), 117–128.

[18] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 861–864.

[19] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 conference of the north american chapter of the association for computational linguistics: Human language technologies (volume 1: Long papers)*. 7036–7050.

[20] Wenqi Jiang, Suvinay Subramanian, Cat Graves, Gustavo Alonso, Amir Yazdanbakhsh, and Vidushi Dadu. 2025. Rago: Systematic performance optimization for retrieval-augmented generation serving. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 974–989.

[21] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems* 44, 1 (2025), 1–27.

[22] Jiajie Jin, Yutao Zhu, Zhicheng Dou, Guanting Dong, Xinyu Yang, Chenghao Zhang, Tong Zhao, Zhao Yang, and Ji-Rong Wen. 2025. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. In *Companion Proceedings of the ACM on Web Conference 2025*. 737–740.

[23] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-scale similarity search with GPUs. *IEEE transactions on big data* 7, 3 (2021), 535–547.

[24] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[25] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*. 6769–6781.

[26] Junkyum Kim and Divya Mahajan. 2026. VectorLiteRAG: Latency-aware and fine-grained resource partitioning for efficient RAG. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–15.

[27] Ravi Kumar and Sergei Vassilvitskii. 2010. Generalized distances between rankings. In *Proceedings of the 19th international conference on World wide web*. 571–580.

[28] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[30] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[31] Conglong Li, Minjia Zhang, David G Andersen, and Yuxiong He. 2020. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2539–2554.

[32] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. 2025. Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–27.

[33] Chien-Yu Lin, Keisuke Kamahori, Yiyu Liu, Xiaoxiang Shi, Madhav Kashyap, Yile Gu, Rulin Shao, Zihao Ye, Kan Zhu, Rohan Kadekodi, et al. 2025. TeleRAG: Efficient retrieval-augmented generation inference with lookahead retrieval. *arXiv preprint arXiv:2502.20969* (2025).

[34] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, et al. 2025. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *Advances in Neural Information Processing Systems* 38 (2025), 54358–54385.

[35] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, et al. 2025. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665* (2025).

[36] Jianan Lu, Asaf Cidon, and Michael Freedman. 2026. When Enough is Enough: Rank-Aware Early Termination for Vector Search. *Proceedings of Machine Learning and Systems* 8 (2026), 108–122.

[37] Yu A Malkov and Dmitry A Yashunin. 2020. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2020), 824–836.

[38] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[39] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vattention: Dynamic memory management for serving llms without pagedattention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1133–1150.

[40] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2026. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* 22, 4 (2026), 1–38.

[41] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*. 2383–2392.

[42] Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2025. Metis: fast quality-aware rag systems with configuration adaptation. In *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles*. 606–622.

[43] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search. *arXiv preprint arXiv:2105.09613* (2021).

[44] Chihiro Taguchi, Seiji Maekawa, and Nikita Bhutani. 2025. Efficient Context Selection for Long-Context QA: No Tuning, No Iteration, Just Adaptive-k. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 20116–20141.

[45] Alex Tamkin, Miles Brundage, Jack Clark, and Deep Ganguli. 2021. Understanding the capabilities, limitations, and societal impact of large language models. *arXiv preprint arXiv:2102.02503* (2021).

[46] Juraj Vladika and Florian Matthes. 2025. On the influence of context size and model choice in retrieval-augmented generation systems. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 6739–6751.

[47] Jiahao Wang, Weiyu Xie, Mingxing Zhang, Boxin Zhang, Jianwei Dong, Yuening Zhu, Chen Lin, Jingqi Tang, Yaochen Han, Zhiyuan Ai, et al. 2026. From prefix cache to fusion rag cache: Accelerating llm inference in retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–28.

[48] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533* (2022).

[49] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating graph indexing for anns on modern cpus. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–29.

[50] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978.

[51] William Webber, Alistair Moffat, and Justin Zobel. 2010. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems (TOIS)* 28, 4 (2010), 1–38.

[52] Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2024. Infllm: Training-free long-context extrapolation for llms with an efficient context memory. *Advances in neural information processing systems* 37 (2024), 119638–119661.

[53] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, et al. 2023. Spfresh: Incremental in-place update for billion-scale vector search. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 545–561.

[54] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[55] Kaixiang Yang, Hongya Wang, Bo Xu, Wei Wang, Yingyuan Xiao, Ming Du, and Junfeng Zhou. 2021. Tao: A learning framework for adaptive nearest neighbor search using static features only. *arXiv preprint arXiv:2110.00696* (2021).

[56] Zehai Yang and Shimin Chen. 2026. RAIRS: Optimizing Redundant Assignment and List Layout for IVF-Based ANN Search. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–27.

[57] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 2369–2380.

[58] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.

[59] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*. 521–538.

[60] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[61] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 193–210.